Cuestionario: Fundamentos y Configuración de Apache

Módulo: Despliegue de Aplicaciones Web (0614) — 2.º DAW

Alumno: Raúl Sal Romeo **Fecha:** 09/11/2026

Respuestas claras y justificadas a las cuestiones técnicas de la Unidad 2.

1. Rutas y estructura predeterminada

Al instalar Apache en un sistema GNU/Linux Debian, ¿en qué directorio exacto se crea por defecto la página web inicial (`index.html`) y qué archivo de configuración contiene las directivas predeterminadas para dicha página?

Directorio: la página inicial se crea en `/var/www/html/index.html`. Ese directorio es el valor por defecto de la directiva `DocumentRoot`, es decir, la raíz del árbol de documentos que Apache publica.

Archivo de configuración: las directivas de ese sitio predeterminado están en `/etc/apache2/sites-available/000-default.conf`, que define el `DocumentRoot /var/www/html` y se activa mediante el enlace simbólico correspondiente en `/etc/apache2/sites-enabled/`. La configuración global del servidor vive en `/etc/apache2/apache2.conf`.

2. Concepto técnico de página estática

Justifica por qué una página actualizada manualmente con cierta frecuencia (por ejemplo, un manual de instrucciones) sigue considerándose «estática» y qué requisito técnico marca la diferencia con una página «dinámica».

Sigue siendo estática porque su contenido está almacenado tal cual en un archivo (HTML, CSS, imágenes) y el servidor lo entrega **sin modificarlo ni ejecutar código** en cada petición. Que el autor la edite a mano y la vuelva a subir con cierta periodicidad no cambia su naturaleza: para todos los usuarios y en todas las peticiones se devuelve exactamente el mismo contenido guardado en disco.

El requisito técnico que marca la diferencia es que la página dinámica se **genera en el momento de la petición** mediante la ejecución de código en el servidor (PHP, Python, Java, etc.), que puede consultar una base de datos o usar datos de la petición. Es decir, la clave no es la frecuencia de actualización, sino **quién y cuándo se produce el contenido**: archivo preexistente (estática) frente a contenido generado por el servidor en cada solicitud (dinámica).

3. Impacto del contenido dinámico

¿Qué consecuencias tiene para un servidor web ofrecer páginas dinámicas en lugar de estáticas en términos de consumo de recursos, administración y seguridad?

- **CPU y memoria:** cada petición dinámica obliga a arrancar o reutilizar un intérprete (PHP, Tomcat…), ejecutar código y, con frecuencia, consultar una base de datos. El coste por petición es mucho mayor que servir un archivo estático, que apenas consume E/S de disco y puede cachearse con facilidad. Por tanto, se atienden menos peticiones simultáneas con el mismo hardware.
- **Administración:** hay que instalar, configurar y actualizar los módulos y runtimes necesarios (por ejemplo `libapache2-mod-php`), gestionar permisos, bases de datos, sesiones y logs más complejos. El mantenimiento es más costoso que el de un simple directorio de archivos.
- **Seguridad:** aumenta la superficie de ataque al haber más software expuesto (intérpretes, módulos, frameworks, base de datos). Aparecen riesgos propios del código (inyección SQL, XSS, subida de archivos, gestión de sesiones), y es crítico mantener los módulos actualizados y validar todas las entradas.

4. Comunicaciones HTTP vs HTTPS

Explica las diferencias fundamentales en el flujo de información entre navegador y servidor con HTTP frente a HTTPS e indica los puertos TCP asignados por defecto a cada uno en Apache.

HTTP envía la petición y la respuesta en **texto plano**: cualquier intermediario de la red puede leer o modificar las cabeceras, las cookies o el cuerpo del mensaje. No garantiza confidencialidad, integridad ni autenticidad del servidor. En Apache escucha por defecto en el **puerto TCP 80** (`Listen 80`, `<VirtualHost *:80>`).

HTTPS es HTTP encapsulado en una capa **TLS**. Antes de intercambiar datos se realiza el *handshake*: el servidor presenta su certificado, el cliente lo valida contra una autoridad de certificación de confianza y ambos acuerdan las claves de sesión. A partir de ahí todo el tráfico viaja cifrado, lo que aporta confidencialidad, integridad y autenticación del servidor. En Apache se habilita con `SSLEngine on` y escucha por defecto en el **puerto TCP 443** (`Listen 443`, `<VirtualHost *:443>`).

5. Certificados y confianza

Si usamos un certificado creado por el propio servidor para una página pública en Internet, ¿cómo reaccionará el navegador del cliente al recibirlo y cuál es el motivo técnico?

El navegador mostrará una **advertencia de conexión no segura** (por ejemplo, «*Tu conexión no es privada*» o `NET::ERR_CERT_AUTHORITY_INVALID`) y exigirá al usuario aceptar una excepción para continuar.

El motivo es que un certificado autofirmado está firmado por el propio servidor y no por una **Autoridad de Certificación (CA) de confianza** incluida en el almacén del sistema operativo o del navegador. Al no poder construir una cadena de confianza hasta una CA reconocida, el cliente no puede verificar que el servidor sea realmente quien dice ser, por lo que trata el certificado como no fiable.

6. Estándar y cabeceras MIME

Define la utilidad del estándar MIME, indica mediante qué campo de la cabecera de respuesta HTTP avisa el servidor del tipo de datos y qué hace el cliente si el archivo carece de tipo MIME.

MIME (*Multipurpose Internet Mail Extensions*) es el estándar que identifica el tipo de contenido mediante la notación `tipo/subtipo` (por ejemplo, `text/html`, `image/png`, `application/pdf`). Su utilidad es que el navegador sepa **cómo interpretar y presentar** los datos recibidos, independientemente de la extensión del archivo.

El servidor lo comunica con el campo de cabecera de respuesta `Content-Type`. Si un archivo no tiene tipo MIME especificado, el cliente no puede saber cómo tratarlo y normalmente lo considera `application/octet-stream`, es decir, datos binarios genéricos: en lugar de mostrarlo, ofrece **descargarlo** o pregunta al usuario qué aplicación debe abrirlo.

7. Directivas MIME en Apache

Diferencia el comportamiento y la finalidad de las directivas `DefaultType` y `ForceType`.

`DefaultType` establece el tipo MIME **por defecto o de reserva**: solo se aplica cuando Apache no puede deducir el tipo del archivo por su extensión u otras reglas MIME (`AddType`, `TypesConfig`). No afecta a los archivos cuyo tipo sí se conoce.

`ForceType` **impone obligatoriamente** un tipo MIME concreto a todos los archivos del ámbito donde se declara (directorio, `<Location>`, `<Files>`), ignorando la extensión. Es útil, por ejemplo, para servir todos los ficheros de una carpeta como `text/plain` o para que un script sin extensión se trate como `application/x-httpd-cgi`.

En resumen: `DefaultType` = valor de reserva para tipos desconocidos; `ForceType` = valor impuesto que prevalece sobre la extensión.

8. Concepto base de VirtualHosts

¿Para qué sirven los hosts virtuales y cómo interactúan las directivas globales de `apache2.conf` con cada virtualhost?

Los **hosts virtuales** permiten alojar **varios sitios web independientes en un mismo servidor** (una misma IP y puerto), cada uno con su propio `DocumentRoot`, nombre de dominio, logs y directivas. Apache elige el host virtual adecuado según la IP/puerto de la conexión y, sobre todo, la cabecera `Host` de la petición, comparándola con `ServerName` y `ServerAlias`.

Las directivas globales de `apache2.conf` actúan como **configuración común heredada**: se aplican a todos los sitios salvo que un virtualhost las sobrescriba en su propio ámbito. El virtualhost solo puede redefinir aquellas directivas que son válidas en ese contexto, y lo que no redefine sigue tomándose del ámbito global. Por eso lo global define el comportamiento general del servidor y cada virtualhost particulariza su sitio.
